DATA ENGINEERING & CLOUD INFRASTRUCTURE

MédiRelais

Moving medicines that are about to expire to the hospital that need them in time

Manel Rahali · Hadiqa Javaid · Soumaya Atoui

November, 6th 2026

THE PROBLEM

Hospitals throw away medicines that another hospital nearby is missing

€96M

of medicines bought by the network each year

€1.44M

thrown away each year, because it expired on a shelf

€576k

a year saved, our target: 40% of that waste

Each pharmacy only sees its own stock. MédiRelais connects the hospital pharmacies and suggests transfers every morning.

WHAT MÉDIRELAIS DOES

Two questions answered for each pharmacist, every morning

USE CASE 1 · CORE · DAILY 06:00

What is about to expire, what is it worth, and who can use it in time?

Value at risk and transfer suggestions

USE CASE 2 · WEEKLY

How many days will my stock last at the current pace?

Days of cover and over-stock alerts, to order less

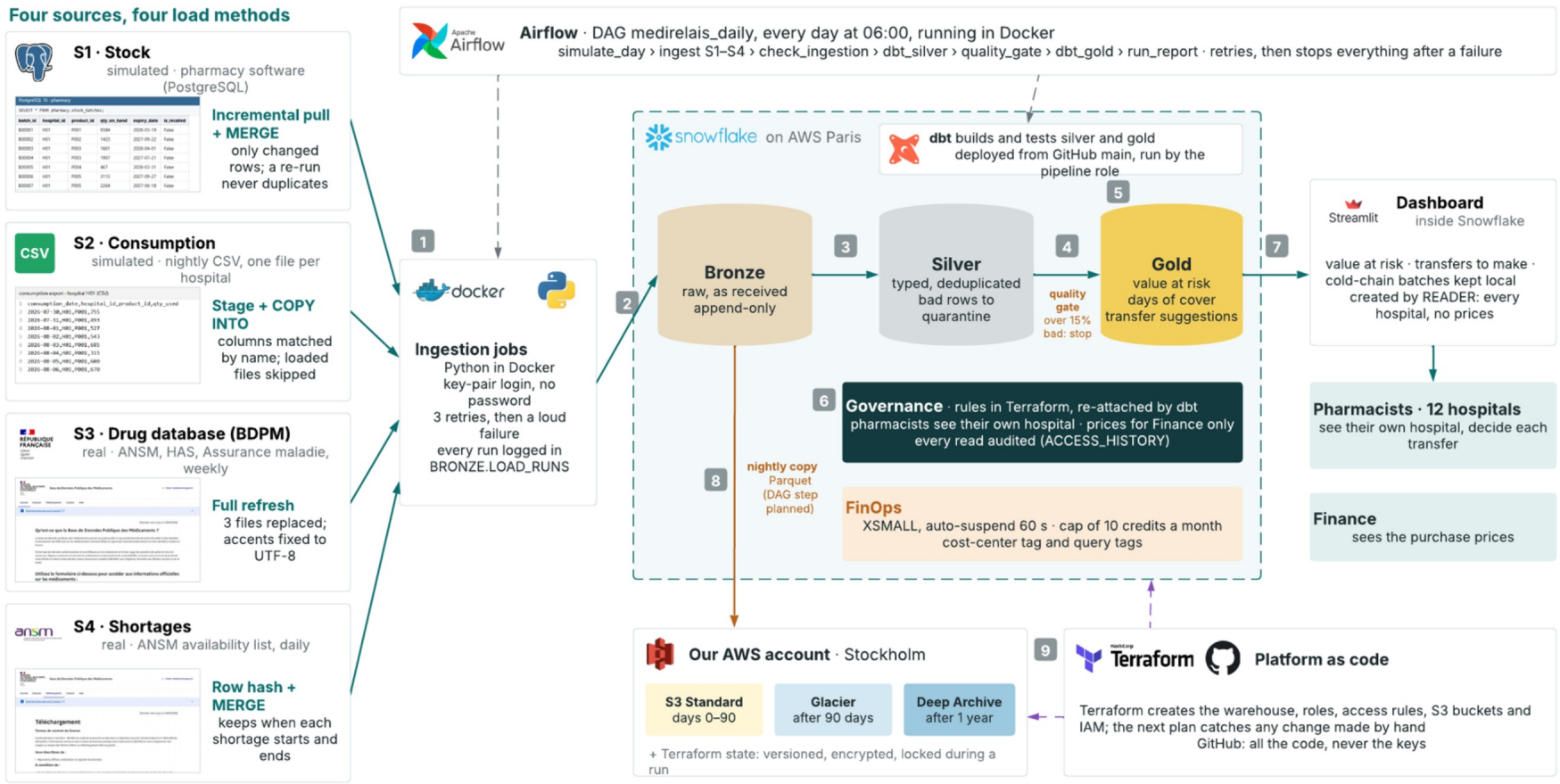
€1,500 a month per hospital

€216k a year, subscription only for 12 pilot hospitals

Decision support only: the system suggests, the pharmacists decide and organise the transfer.

THE PIPELINE AT A GLANCE

Four sources become one transfer list by 06:00



1 extract the 4 sources

2 load bronze, raw

3 clean into silver

4 quality gate

5 compute gold

6 access rules

7 serve the dashboards

8 archive to S3

9 platform as code

THE STACK

Each tool has one job in the pipeline

PostgreSQL

the hospitals' stock system (simulated)

Python · Docker

one ingestion job per source

Airflow

runs the chain at 06:00, stops on bad data

Snowflake

stores and computes bronze, silver, gold

dbt

cleans, computes and tests

Streamlit

3 dashboards inside Snowflake

Terraform

builds Snowflake and AWS from code

AWS S3 · GitHub

archive and state · all the code, never keys

PART 1

Sources, ingestion and orchestration

Where the data comes from, how each source reaches Snowflake, and who runs it every morning

PART 1 · SOURCES

One load method per source, chosen by how it arrives

S1 hospital stock

CSV S2 consumption

S3 drug database

S4 shortages

Source	Real?	Arrives as	Method	Used for
S1 Stock	simulated	Postgres database	Incremental pull + MERGE	batches, expiry
S2 Consumption	simulated	CSV per hospital, nightly	Stage + COPY INTO	daily use
S3 Drug database	real	3 files, weekly	Full refresh	national codes, molecules
S4 Shortages	real	1 file, daily	Row hash + MERGE	shortage priority

Every run is logged in BRONZE.LOAD_RUNS. Code: ingestion/ on GitHub.

PART 1 · SIMULATOR

Data from 12 hospitals

Hospitals Data

Stock scanned in the pharmacy software, one consumption file per hospital each night. No form, no manual entry.

Real-life unperfect data

Dates in 4 formats, “120 units”, “ h03 ”, “N/A”, unknown codes, duplicates, a column renamed on day 3

An answer key for every mistake

landing/issues/day_N.json, checked against silver’s quarantine

PostgreSQL 16 · pharmacy

```
SELECT * FROM pharmacy.stock_batches;
```

batch_id	hospital_id	product_id	qty_on_hand	expiry_date	is_recalled
B00001	H01	P001	9384	2028-03-19	False
B00002	H01	P002	1423	2027-09-22	False
B00003	H01	P003	1601	2028-04-01	False
B00004	H01	P003	1907	2027-07-21	False
B00005	H01	P004	467	2028-03-31	False
B00006	H01	P005	3113	2027-09-27	False
B00007	H01	P005	2264	2027-06-18	False

consumption export · hospital H01 (CSV)

```
consumption_date,hospital_id,product_id,qty_used
2026-07-30,H01,P001,755
2026-07-31,H01,P001,493
2026-08-01,H01,P001,527
2026-08-02,H01,P001,543
2026-08-03,H01,P001,685
2026-08-04,H01,P001,315
2026-08-05,H01,P001,600
2026-08-06,H01,P001,670
```

PART 1 · S1 STOCK

Only the rows that changed are pulled, then merged

1 · Watermark
latest updated_at already in Snowflake

›

2 · Pull
Postgres rows changed after it

›

3 · Temp table
the changed rows land in Snowflake

›

4 · MERGE on key
exists: update · new: insert

PostgreSQL 16 · pharmacy

```
SELECT * FROM pharmacy.stock_batches;
```

batch_id	hospital_id	product_id	qty_on_hand	expiry_date	is_recalled
B00001	H01	P001	9384	2028-03-19	False
B00002	H01	P002	1423	2027-09-22	False
B00003	H01	P003	1601	2028-04-01	False
B00004	H01	P003	1907	2027-07-21	False
B00005	H01	P004	467	2028-03-31	False
B00006	H01	P005	3113	2027-09-27	False
B00007	H01	P005	2264	2027-06-18	False

Re-running never duplicates a row: the MERGE is idempotent

Every row carries _source, _loaded_at and _batch_id, so it traces back to its run

Day 3 run: 62 rows changed in Postgres, 62 merged. Code: ingestion/jobs/ingest_stock_postgres.py

PART 1 · S2 CONSUMPTION

Nightly files go to a stage, then one COPY INTO loads them

1 · Nightly files
dt=2026-10-30/H01.csv … H12.csv

›

2 · PUT to stage
Snowflake internal stage RAW_STAGE

›

3 · COPY INTO
CONSUMPTION_RAW, columns matched by name

consumption export · hospital H01 (CSV)

```
consumption_date,hospital_id,product_id,qty_used
2026-07-30,H01,P001,755
2026-07-31,H01,P001,493
2026-08-01,H01,P001,527
2026-08-02,H01,P001,543
2026-08-03,H01,P001,685
2026-08-04,H01,P001,315
2026-08-05,H01,P001,600
2026-08-06,H01,P001,670
```

Matched by name: a column renamed on day 3 lands in its own column instead of breaking the load

COPY remembers loaded files: a second run loads nothing; only --force reloads (incident 2)

Day 3 run: 12 new files, 315 lines loaded. Code: ingestion/jobs/ingest_consumption_files.py

PART 1 · S3 DRUG DATABASE

Three real government files, fully replaced each week

CIS_bdpm
one line per medicine: CIS code, name, form

CIS_CIP_bdpm
one line per box: CIP13 barcode, price

CIS_COMPO_bdpm
which molecule is in which medicine



Day 3 run: 69,187 rows. Source: base-donnees-publique.medicaments.gouv.fr (ANSM, HAS, Assurance maladie)

PART 1 · S4 SHORTAGES

A fingerprint of each line shows what is new and what ended

ansm
Agence nationale de sécurité du médicament et des produits de santé

New hash

new shortage: insert, first_seen_at = today

Known hash

still in shortage: update last_seen_at

Hash gone

shortage ended: still_listed = false

The full list is republished daily and only a few lines change: a SHA-256 fingerprint per line and a MERGE keep the history of every shortage, never a duplicate. **Gold ranks medicines in shortage first.**

Day 3 run: 706 lines read, 0 new (nothing changed since the previous run). Code: ingestion/jobs/ingest_shortages.py

PART 1 · INGESTION JOBS

Every job connects the same way and logs each run

Key-pair login

private key on the laptop, public key on the Snowflake user; no password, never on GitHub

A batch id per run

written on every row it loads: any row traces back to its run

A log line per run

BRONZE.LOAD_RUNS: job, status, rows read, rows loaded, error

Retries, then a loud failure

3 attempts; a failed job exits with an error code that Airflow sees

Code: ingestion/common.py. Settings in ingestion/.env and the key in ingestion/keys/, both excluded from Git.

PART 1 · MATCHING

Hospitals name medicines differently, so we match by molecule

“Doliprane 1 g”
brand name at one hospital

“Paracétamol Biogaran”
generic at another



SAME MOLECULE

paracetamol

same national codes (CIS), linked to the shortage list (S4)

Composition file finds brand names

Name match finds generics

Accents normalised first

dbt model silver_product_reference, macros normalize_text and contains_word. CIS = one code per medicine; CIP13 = the barcode on the box.

PART 1 · ORCHESTRATION

Airflow replaced a job that ran dbt alone, with no checks

BEFORE · SNOWFLAKE TASK

- Ran dbt at 06:00, nothing else
- Ingestion launched by hand
- No check that the data arrived
- Broken data still reached gold

NOW · DAG MEDIRELAIS_DAILY

- Runs the whole chain in order at 06:00
- Retries a failed step, then stops what follows
- Checks loads and quality before gold
- Keeps the history of every run

The Snowflake task DAILY_PIPELINE is suspended, so the pipeline never runs twice.

PART 1 · ORCHESTRATION

Airflow runs in Docker: two containers, one command

docker **The laptop**

airflow-db
Postgres 16: Airflow's memory (runs, task states)

airflow 3.1.8
scheduler, DAG processor, web UI on localhost:8080

Mounted, not copied: dags/, tasks/, ingestion/ from the repo, so a git pull is enough



Snowflake
over the internet, key-pair login

Hospitals' Postgres
another container, port 5433

Public drug database
downloads for S3 and S4

Start: cd airflow && docker compose up -d --build · Stop: docker compose down (run history kept)

PART 1 · ORCHESTRATION

Airflow gives the orders; Snowflake does the work

Airflow task
a BashOperator runs a Python script

›

Python script
connects with the private key

›

Snowflake
runs the SQL, or dbt with EXECUTE DBT PROJECT

›

Answer
SUCCESS true or false, plus the dbt log

Exit code 0: task green
the next tasks start

Exit code 1: task red
retried if retries are left, then every task after it is upstream_failed

dbt runs inside Snowflake with the role MEDIRELAIS_PIPELINE, deployed from GitHub main. Script: airflow/tasks/run_dbt.py

PART 1 · THE DAG

LIVE

10 tasks in 7 steps, every morning at 06:00



4 loads in parallel after the day is simulated

2 checks can stop the run: check_ingestion, quality_gate

run_report prints the day's transfers

simulate_day >> ingestion >> check_ingestion >> dbt_silver >> quality_gate >> dbt_gold >> run_report

PART 1 · THE DAG

A few settings make the DAG safe to run every day

Setting	Value	Why
schedule	0 6 * * * (Paris)	ready before pharmacies open at 08:00
catchup	False	do not replay the days missed
max_active_runs	1	never two runs at the same time
retries	2 ingestion · 1 dbt · 0 checks	network glitches fix themselves; a check would not
execution_timeout	20 minutes	a stuck task fails instead of blocking
params	day, incident	manual runs: replay a day, simulate a corrupted day

PART 1 · THE TWO CHECKS

Two checks stop bad or missing data before the dashboard

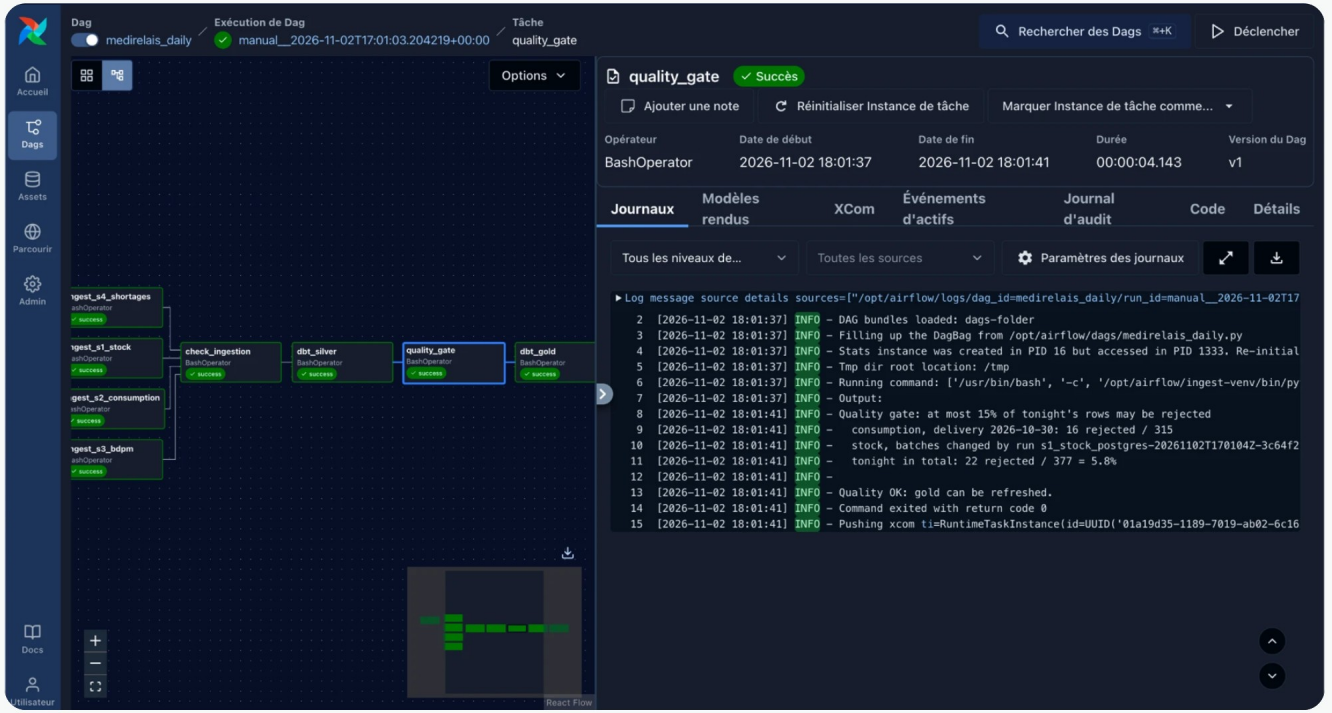
check_ingestion · after the 4 loads

each job's latest LOAD_RUNS line must be SUCCESS, and consumption must have rows; else dbt does not run

quality_gate · after silver

over 15% of tonight's rows in quarantine: gold is not refreshed, the dashboard keeps yesterday

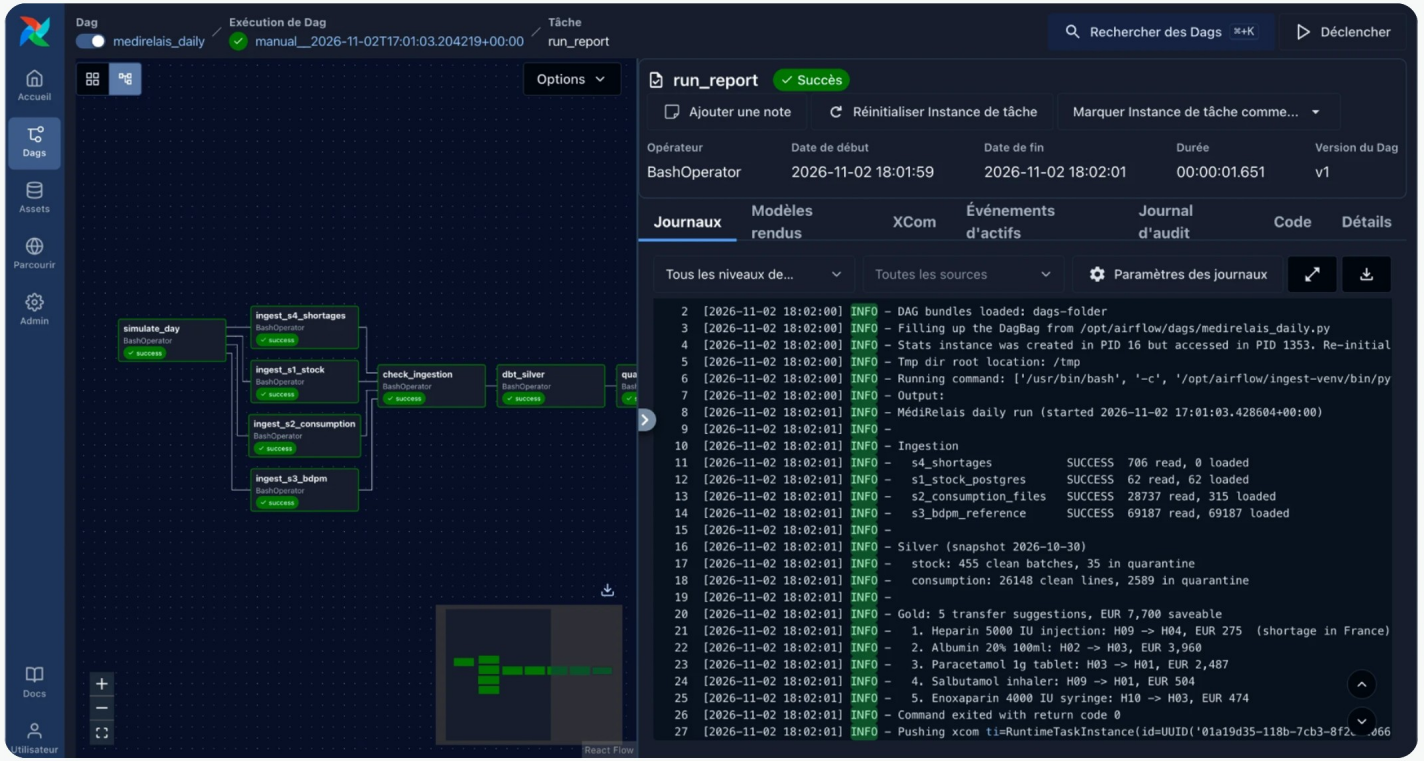
Normal night ≈ 5% · corrupted night ≈ 45%. Duplicates are not errors.



Screenshot: quality_gate on 2 Nov, 22 rejected / 377 = 5.8%, so gold is refreshed. Code: airflow/tasks/

PART 1 · RESULTS

Day 3 ran green end to end: 5 transfers, €7,700 saveable



Day 3 run	Result
S1 stock	62 rows
S2 consumption	315 lines
S3 drug database	69,187 rows
S4 shortages	706 read, 0 new
Quality gate	5.8%
Gold	5 transfers

Run manual__2026-11-02T17:01:03, snapshot 30 Oct 2026, log of run_report.

PART 2

Transformation, quality and dashboards

How silver cleans, how gold decides, how tests and incidents prove it, and what each user sees

PART 2 · SILVER

Silver fixes what it can and quarantines the rest, with a reason

What arrives	What silver does
28/10/2026, 20261028	2026-10-28
“120 units”, “ h03 ”	120, H03
“N/A”, -50, 2099-01-01	quarantine with its reason
Unknown code P999	quarantine: unknown_product
Quantity × 1000 (typo)	quarantine: implausible
Same line twice	keep one: duplicate_row

STOCK AFTER DAY 3

455 clean

35 in quarantine

CONSUMPTION

26,148 clean

2,589 in quarantine

Quarantine counts match the simulator’s answer key. Tables: SILVER.SILVER_*_QUARANTINE.

PART 2 · GOLD

Gold turns the clean data into 6 numbers

Value at risk

quantity × unit cost of batches expiring within 90 days (not expired, narcotic or recalled)

Daily use

average of the last 28 days, only the days actually reported

Days of cover

stock ÷ daily use

Over-stock value

units that will expire before use at the current pace × unit cost

Saveable value

units the receiver can use before expiry × unit cost

Recovery rate

saveable value ÷ value at risk, against a 40% target

dbt models gold_value_at_risk, gold_days_of_cover, gold_transfer_suggestions

PART 2 · TRANSFER RULES

Before moving a batch, gold checks five things in order

1

Expires within 90 days?

No: nothing to do

2

Narcotic or recalled?

Yes: never moves

3

Under 30 days or €200?

Yes: used locally

4

Cold-chain product?

Yes: no proof, never moves

5

Who can use it in time?

Nobody: no suggestion

A batch that passes all five becomes a suggestion. **Ranking: shortage in France first, then the largest saveable value.**

Checked on every build by the dbt test transfers_respect_rules.

PART 2 · CHOOSING THE RECEIVER

The receiver is the hospital that can use most of the batch in time

USABLE QUANTITY

```
MIN( batch quantity , receiver's daily use × days left - its own stock that expires first )
```

Candidates: every other hospital that uses the product

Winner: the largest usable quantity

Nobody can use it in time: no suggestion

Example on day 3: heparin

A batch at H09, a medicine in shortage in France. H04 can use it before expiry, so H09 → H04 is ranked first.

PART 2 · RESULTS

On day 3, 5 suggested transfers would save about €7,700

#	Medicine	From → to	Saveable
1	Heparin 5000 IU (shortage in France)	H09 → H04	€275
2	Albumin 20% 100 ml	H02 → H03	€3,960
3	Paracetamol 1 g (over-stock)	H03 → H01	€2,487
4	Salbutamol inhaler	H09 → H01	€504
5	Enoxaparin 4000 IU	H10 → H03	€474

Simulated days 1–3, snapshot 30 Oct 2026, from run_report on 2 Nov. Cold-chain batches (insulin, epoetin, rituximab) are never suggested.

PART 2 · TESTS

dbt tests stop the build when a business rule breaks

Transfer rules
no narcotic, recalled, expired or cold-chain batch is ever suggested

Planted cases
the acceptance test checks known batches on the latest snapshot

Data quality
quarantine under 15%, values in range, all 12 hospitals reporting

Business order
shortages ranked first; 8 weeks of history before days of cover

Plus key checks on every model: unique, not null, relationships, accepted values.

Code: medirelais_dbt/tests/ (7 test files and a generic accepted_range test), run with every dbt build.

PART 2 · INCIDENTS

LIVE

Three incidents show a failure never corrupts the results

INCIDENT 1

A corrupted day

Trigger: incident = corrupted

Result: quality_gate red, gold not refreshed, the dashboard keeps yesterday

INCIDENT 2

Files loaded twice

Trigger: reload consumption with --force

Result: all green, silver keeps one copy: 0 duplicates

INCIDENT 3

A source is down

Trigger: shortages_url set to a wrong address

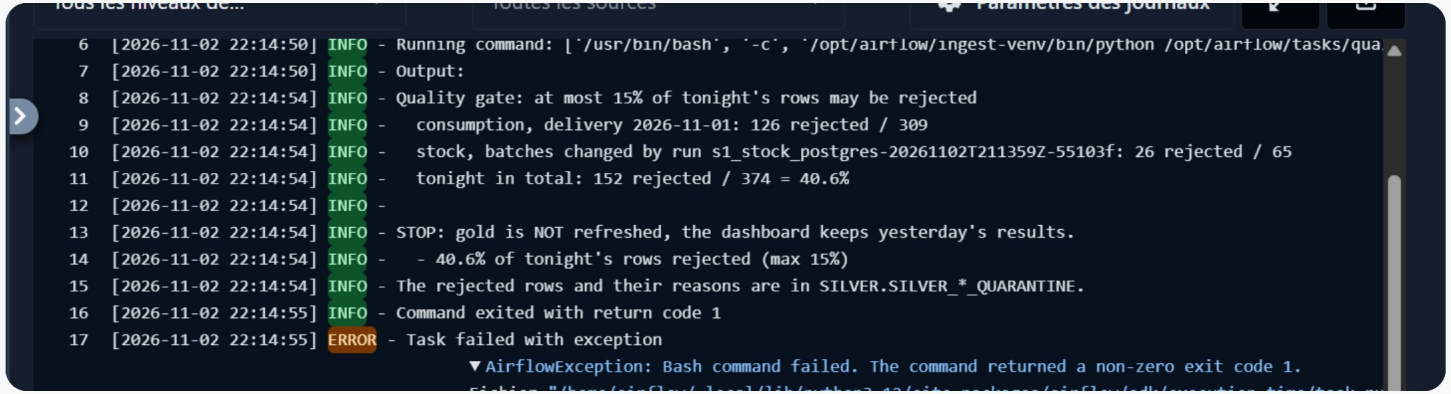
Result: retried twice, then red; green again after the fix and Clear

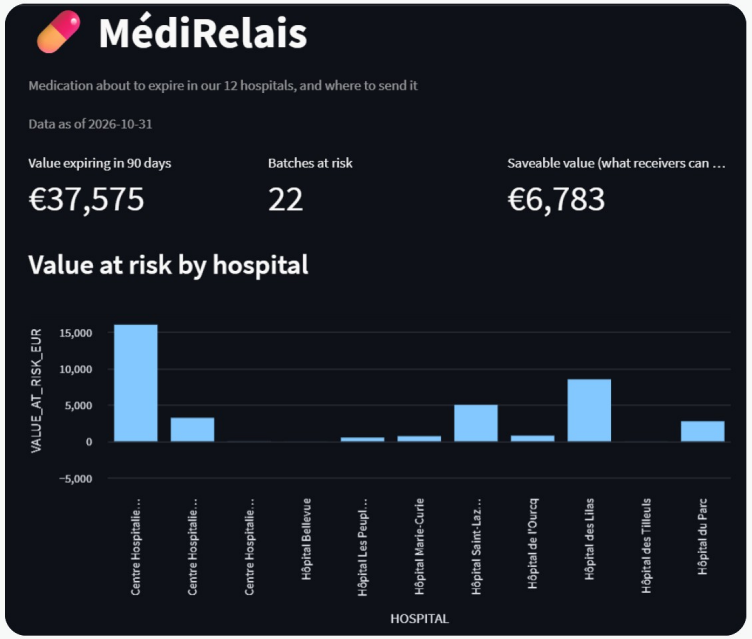
Incidents 1 and 2: Hadiqa. Incident 3: Manel. Reset for a clean demo: reset_demo.py --yes (airflow/README.md)

PART 2 · INCIDENT 1

A corrupted night is stopped at the gate







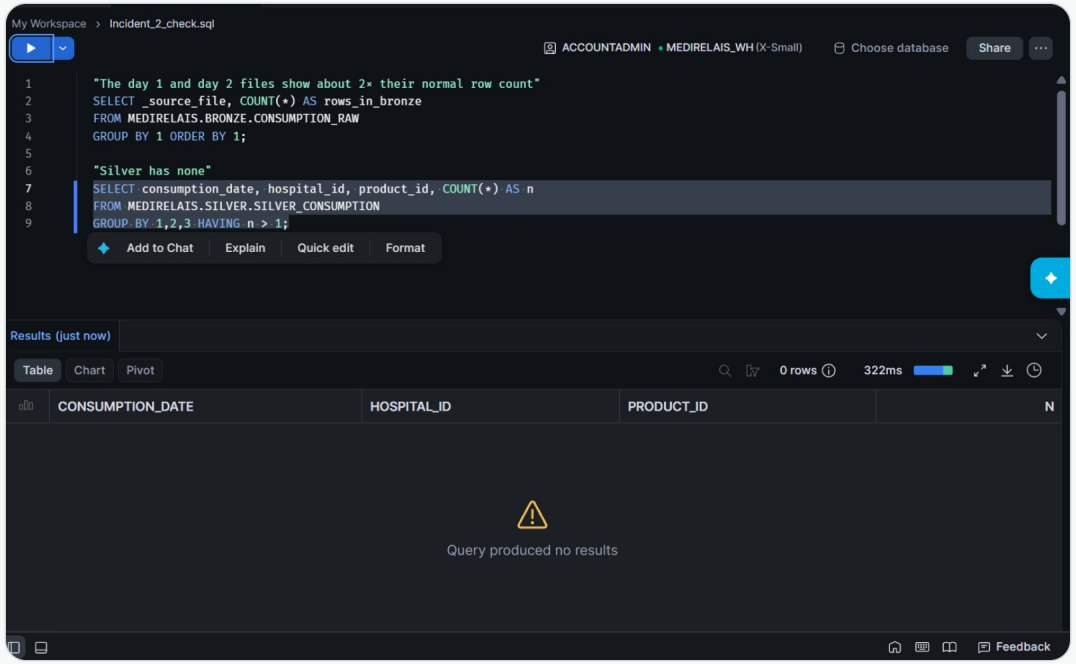
The dashboard still shows the previous day

Rehearsal of 2 Nov: 152 rejected / 374 = 40.6% (max 15%). Proof: docs/Incident 1/

PART 2 · INCIDENTS 2 AND 3

A double load counts once; a dead source stops the run

Incident 2 · files loaded twice



Bronze holds days 1–2 twice; silver returns 0 duplicates

Incident 3 · source down



Retried twice, then red; nothing after it runs

ingest_s4_shortages red after 2 retries, then green after Clear

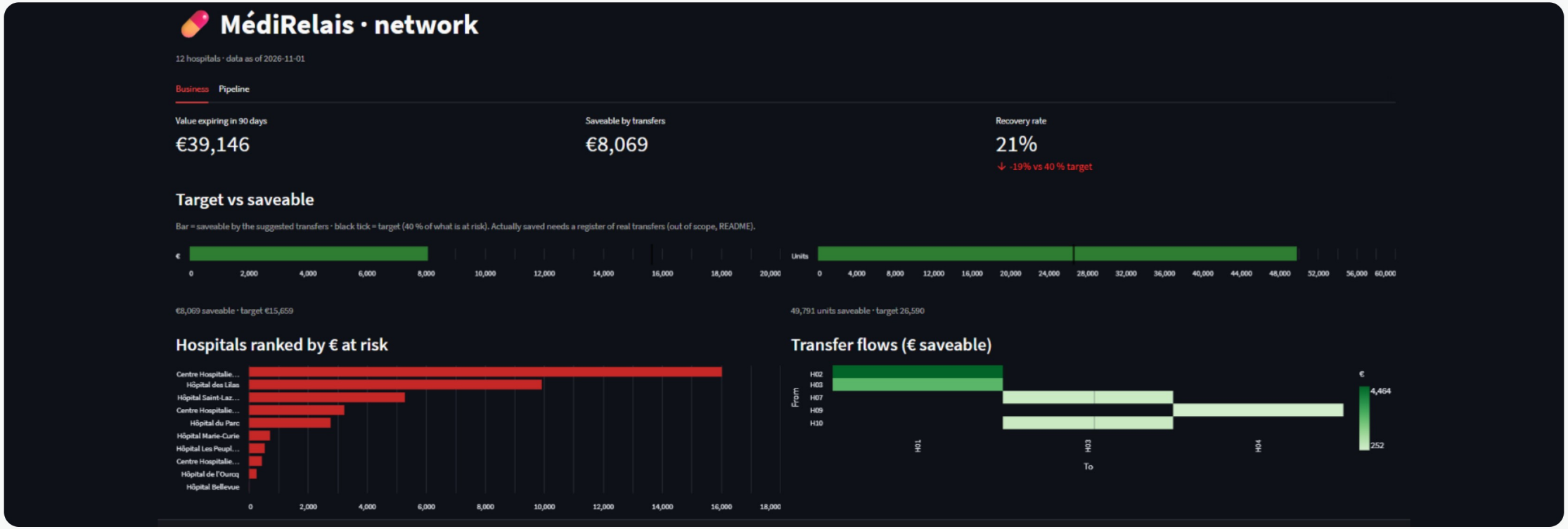
Incident 2 proof: docs/Incident 2/. Incident 3 is Manel's; its screenshot is still to add.

PART 2 · DASHBOARDS

LIVE

The network dashboard shows where the money is

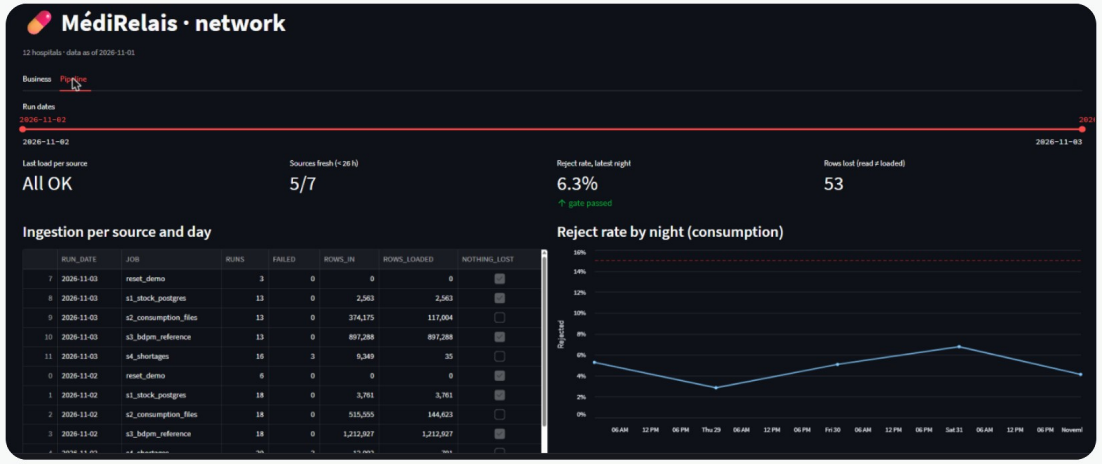
Streamlit



Business tab, live on 4 Nov (data as of 1 Nov). Created by READER: every hospital, no purchase prices. Open live: [link to Snowflake app]

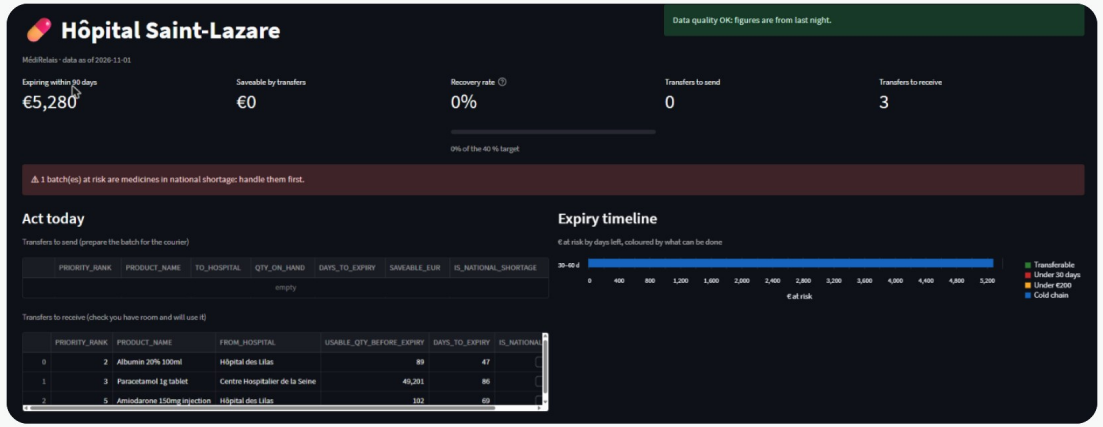
PART 2 · DASHBOARDS

The pipeline tab and each hospital's own app



Pipeline tab

loads per source, reject rate by night, the data-quality tile



Hospital app · H01 and H04

act today, expiry timeline, use first; Snowflake shows its own hospital only

Streamlit in Snowflake, deployed from GitHub main. Results reused for 1 hour: gold only changes at 06:00.

LIVE DEMO

Demo

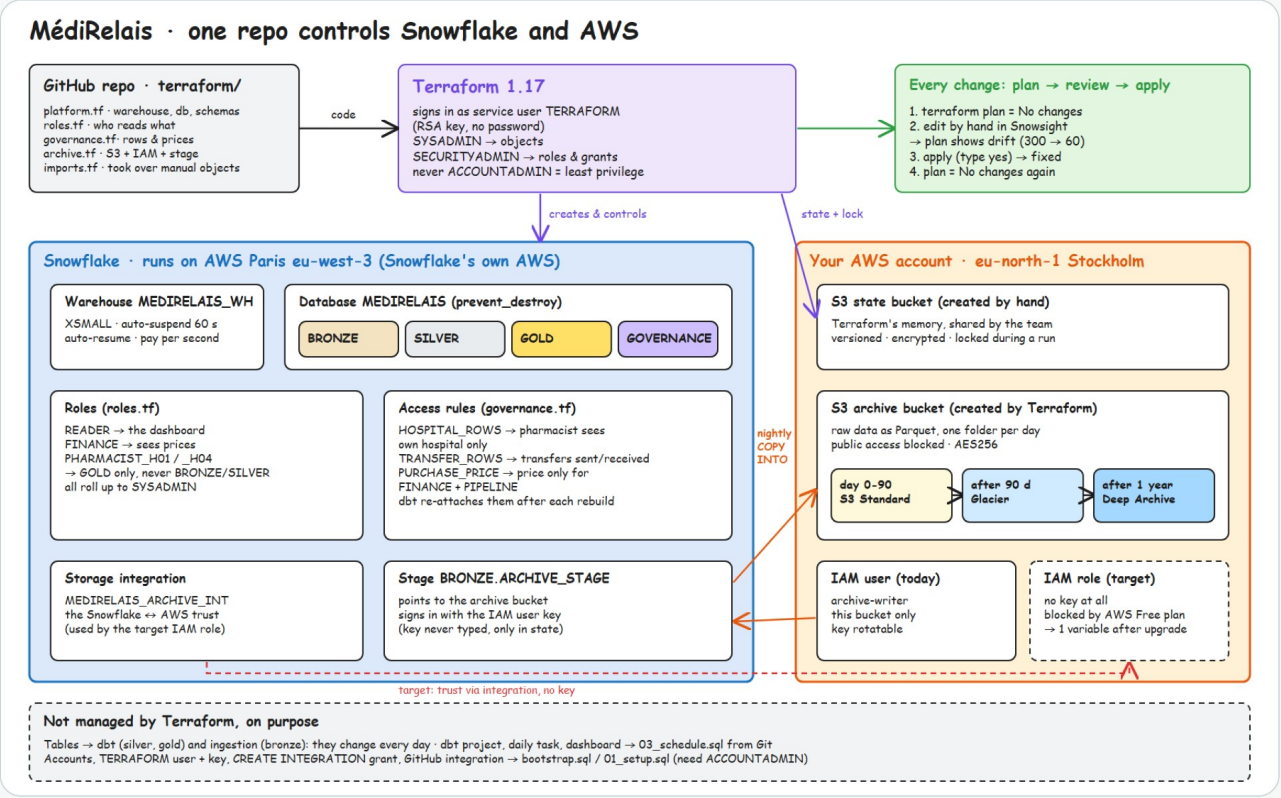
PART 3

Platform, access and costs

Who can change the platform, who sees which data, and what it costs

PART · TERRAFORM

One repo controls Snowflake and AWS



Remote state in S3

versioned, encrypted, locked during a run

Least privilege

key-only service user: SYSADMIN for objects, SECURITYADMIN for grants, never ACCOUNTADMIN

Kept out, on purpose

tables belong to dbt and ingestion; the dbt project and dashboards deploy from Git

Terraform 1.17.1, Snowflake and AWS providers pinned. Code: terraform/ on GitHub.

PART 3 · ARCHIVE AND STATE

Old raw data leaves Snowflake for cheap S3 tiers

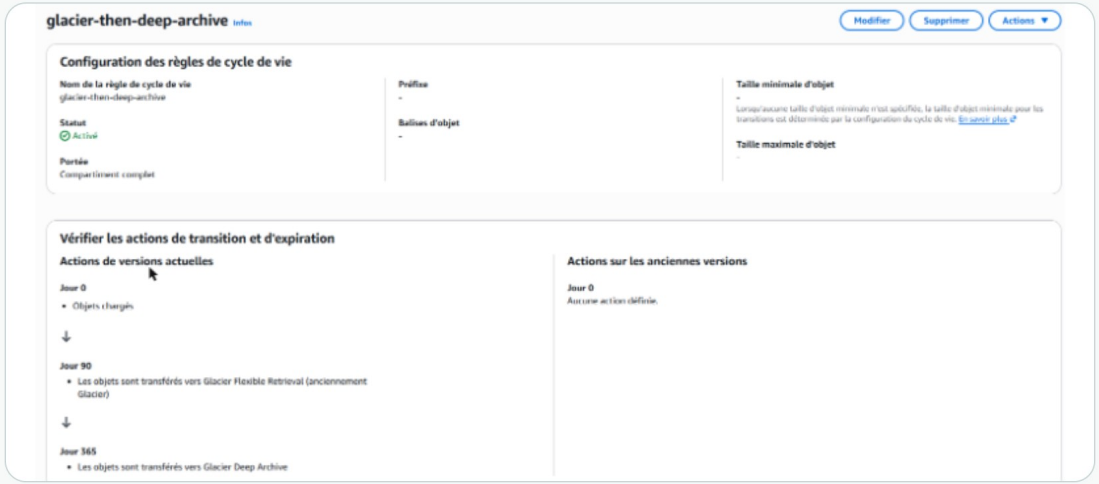
Days 0–90 · S3 Standard
readable at once: replays, audits

›

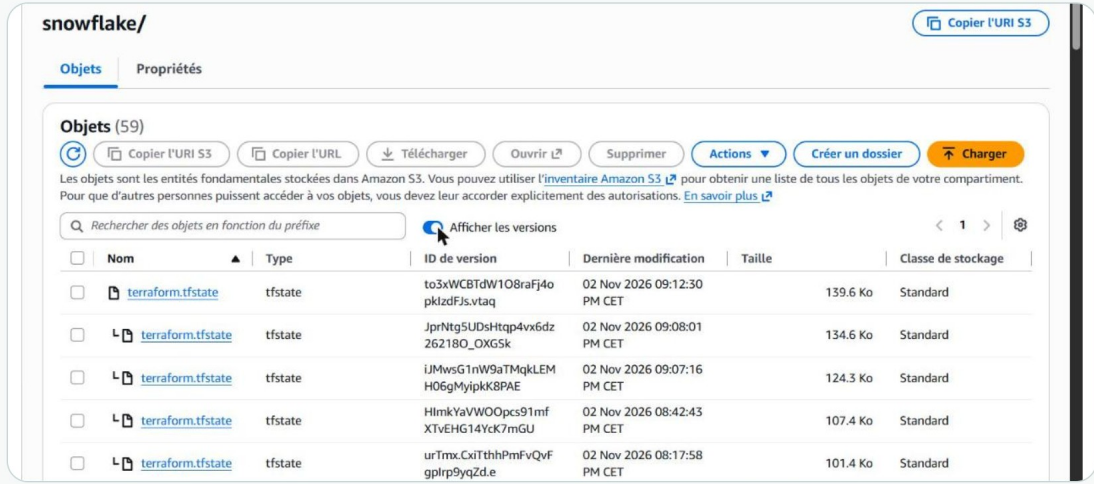
After 90 days · Glacier
cheaper, restore first

›

After 1 year · Deep Archive
storage costs almost nothing



The lifecycle rule in AWS, created by Terraform



Terraform's state: 59 versions kept in S3

Our AWS account is in Stockholm (eu-north-1), Snowflake in Paris: both in the EU. The nightly archive copy is tested; running it as an Airflow step is planned.

PART 3 · GOVERNANCE

LIVE

Each role sees only what its job needs, enforced by Snowflake

Role	Rows in GOLD	Purchase price
Pipeline (builds the tables)	all	visible
Finance	all	visible
Reader: the network dashboard	all	hidden
Pharmacist H01, H04	own hospital only	hidden
Platform admins	all	hidden



Live, as pharmacist H01: the same query returns 1 row, its own, with the price hidden

No patient data: stock and consumption per hospital and product only. Readers are refused on BRONZE. Check: snowflake/04_access_check.sql

PART 3 · GOVERNANCE

The rules live in Terraform and survive every daily rebuild

Terraform
defines 2 row rules and 1 price mask

›

dbt
rebuilds gold every morning

›

Post-hooks
re-attach the rules; cannot rewrite them

›

Each role queries
rows filtered, prices masked

Each dashboard runs as its owner

READER's app sees every hospital, H01's app only H01

The active role, not all roles

an admin testing as H01 sees exactly what H01 sees

Every read is audited

who read which gold table, with which role (ACCESS_HISTORY)

Rules: terraform/governance.tf · re-attached by medirelais_dbt/dbt_project.yml

PART 3 · GOVERNANCE

Every table says who owns it and how sensitive it is

Layer	Owner (steward)	Sensitivity	Retention
Bronze	Manel	public · internal · confidential	kept as received
Silver	Hadiqa	internal · confidential	rebuilt daily
Gold	Hadiqa	internal · confidential	rebuilt daily
Price column	wherever it appears	confidential + masked	—

PII = none, on every table

no patients, no staff: the sensitive data is commercial (negotiated prices)

Survives the nightly rebuild

Terraform tags bronze; dbt re-tags silver and gold after each build

Live in Snowflake, gold value-at-risk table: the price column carries its own **confidential** tag; owner, PII and retention come from the table

	COLUMN_NAME	TAG_NAME	TAG_VALUE	LEVEL
1	UNIT_COST_EUR	SENSITIVITY	confidential	COLUMN
2	UNIT_COST_EUR	DATA_OWNER	hadiqa.javaid	TABLE
3	UNIT_COST_EUR	PII	none	TABLE
4	UNIT_COST_EUR	RETENTION	rebuilt_daily	TABLE

4 tags: DATA_OWNER, SENSITIVITY, PII, RETENTION · terraform/tags.tf · medirelais_dbt/macros/governance_tags.sql

PART 3 · GOVERNANCE

Not everyone is admin: who holds which role, and why

Role hierarchy (each role rolls up to the one above)

ACCOUNTADMIN
2 people: users, billing, monitors

SYSADMIN
owns warehouse, database, schemas

SECURITYADMIN
owns roles and grants

PIPELINE
owns the tables it builds; the only role that writes

READER
gold, no prices

FINANCE
gold + prices

PHARMACIST_Hxx
own hospital

Who	Role and why
Soumaya, Hadiqa	ACCOUNTADMIN: account owners
Manel	SYSADMIN: builds objects, no billing or users
TERRAFORM user	SYSADMIN + SECURITYADMIN, never ACCOUNTADMIN; key pair only
Airflow, dbt	PIPELINE
Dashboards	run as their owner: READER or PHARMACIST_Hxx

	USER_NAME	USER_TYPE	ROLES	KEY_PAIR	PASSWORD
1	HJAVAID	PERSON	ACCOUNTADMIN, SYSADMIN	TRUE	TRUE
2	MRAHALI	PERSON	SYSADMIN	TRUE	TRUE
3	TERRAFORM	SERVICE	SECURITYADMIN, SYSADMIN	TRUE	null
4	satoui	null	ACCOUNTADMIN, MEDIRELAIS_PIPELINE, ORGADMIN	TRUE	FALSE

Role-based (RBAC) + ownership (DAC): whoever owns an object grants on it · live grants and logins: 04_access_check.sql blocks 7–9

PART 3 · FINOPS

See the cost, cap it, then optimise it

Visibility

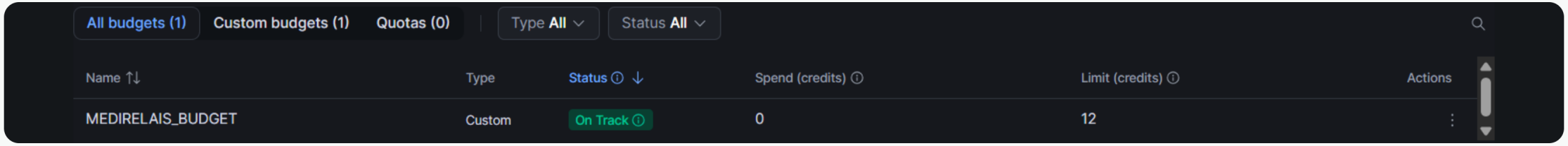
- cost-center tag on warehouse and database
- query tags: dashboards, dbt
- showback per role, since roles map to teams
- cost owner: Soumaya, monthly showback to the team

Control

- budget, all objects tagged medirelais: 12 credits, email if the month will overrun
- monitor, our warehouse: 10 credits (≈ €33), email at 75%, stop at 90%
- auto-suspend after 60 s idle
- AWS budget alert: planned

Optimisation

- query acceleration off
- XSMALL warehouse, right-sized
- dashboards reuse results for 1 hour
- cold data to Glacier, Deep Archive

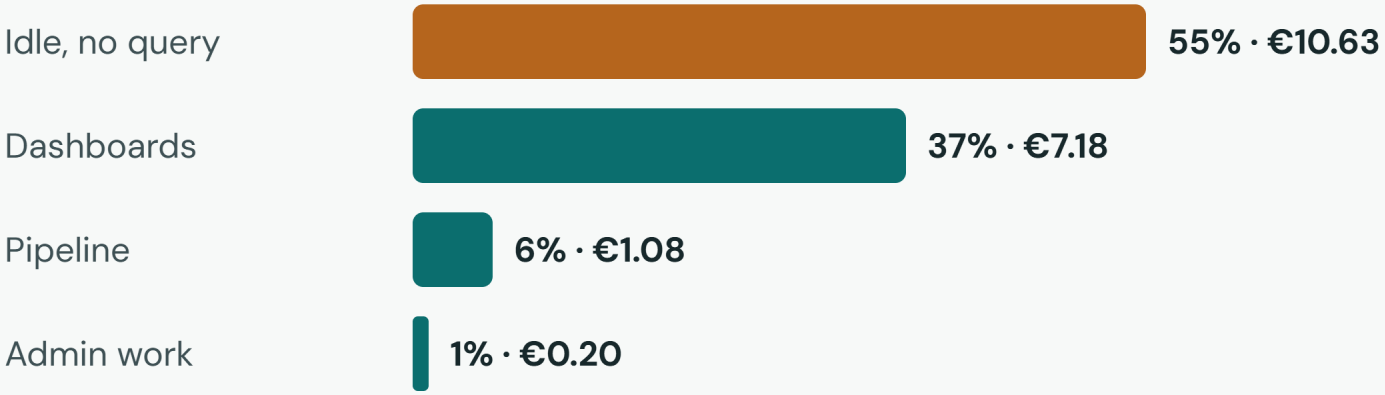


The three FinOps pillars from the course · code: terraform/costs.tf, terraform/platform.tf, snowflake/05_costs.sql

PART · COSTS

Building cost €16 in 30 days; the daily build about €4 a month

Who used the warehouse (≈ 5.7 credits)



€0.14
per dbt build (25–40 s)

€0.001
a month of storage (57 MB)

0.09 MB
copied Paris → Stockholm: billed

Measured 3 Nov 2026 with snowflake/05_costs.sql, last 30 days · $3.90 per credit (Snowflake rate sheet) · €/$ 0.86 assumed

PART 3 · VALUE

Every €1 of platform saves about €470 of medicine

€7,700

saveable by 5 transfers on one day, out of €38,474 at risk

€16.35

platform cost over the last 30 days, mostly development

Next: a separate warehouse for the dashboards

Next: fridge data to unlock cold-chain transfers

Next: showback now, chargeback per team later

€7,700 is one day’s snapshot; €16.35 covers 30 days. An order of magnitude, not a forecast.

Questions

Sources and Airflow: Manel · Transformation and dashboards: Hadiqa · Platform, access and costs: Soumaya